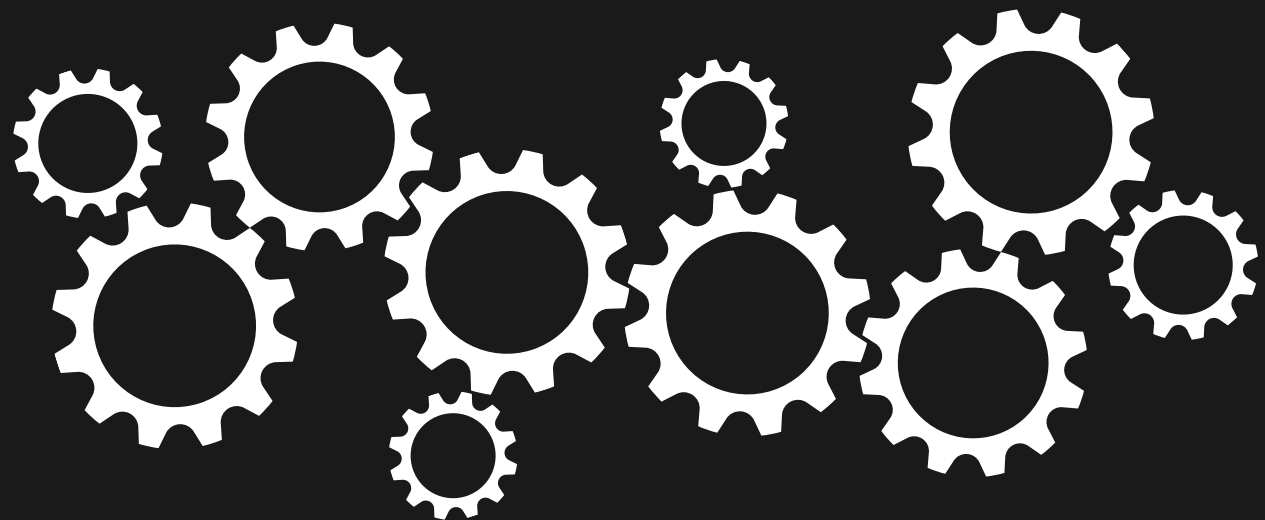
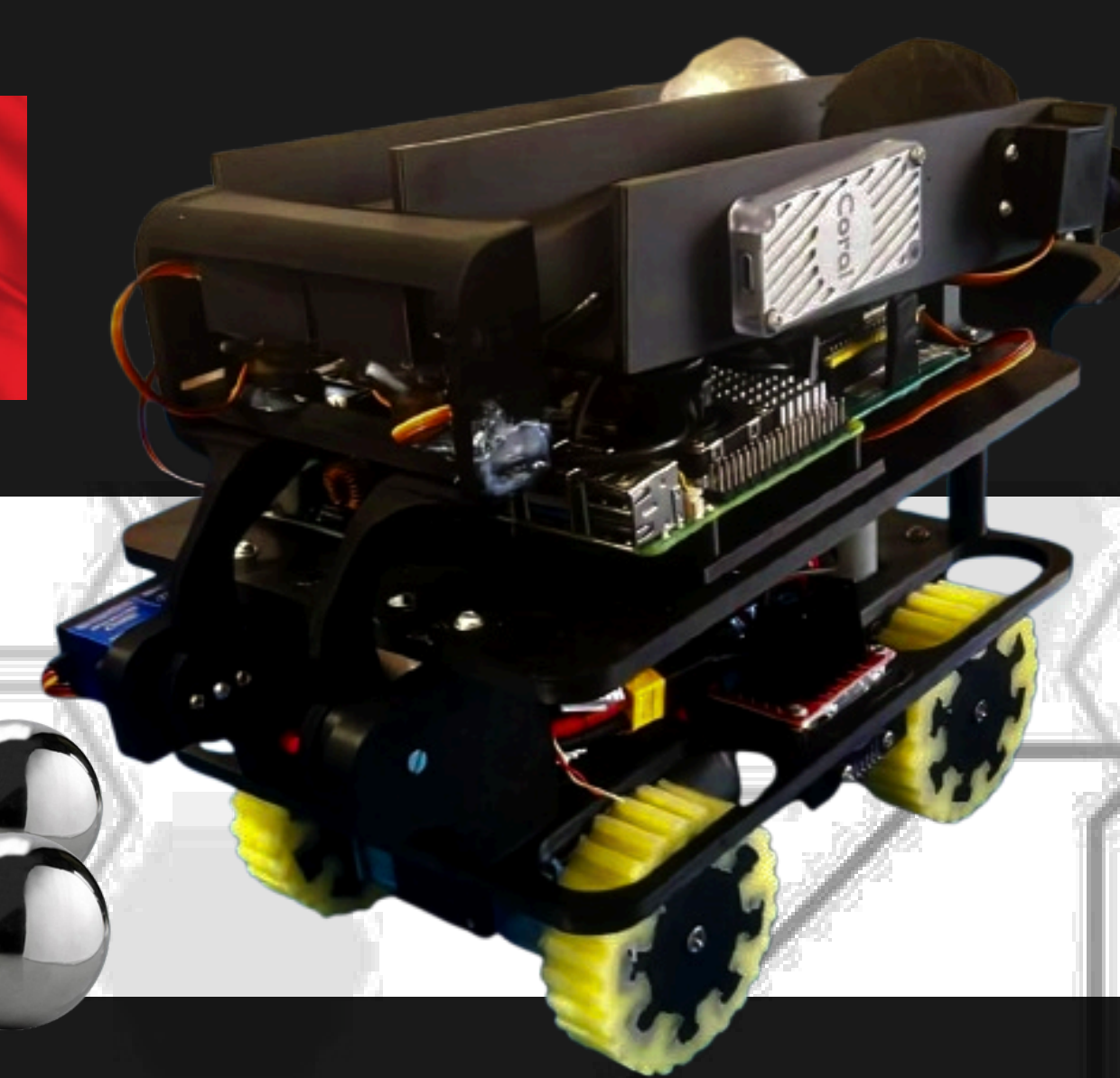




OBLIVION ROBOTICS

Rescue Line RoboCup Junior 2026



HARDWARE

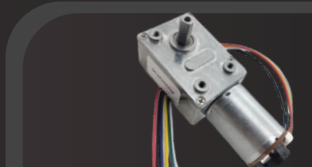
COMPONENTS



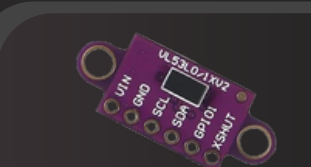
Raspberry Pi 5 8GB



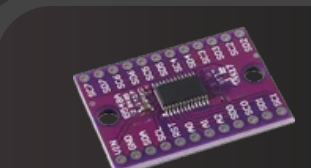
ESP32 ESP-WROOM Module



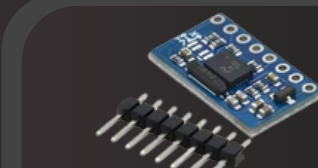
DC Motor 66 RPM (Qty: 4)



ToF VL350X (Qty: 3)



Multiplexer CJMCU-9548



Gyro MPU6050

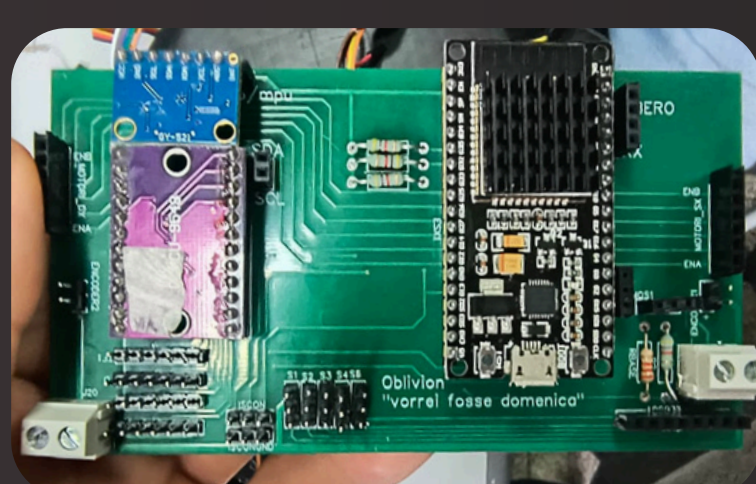


IMX219 Camera Module



Google Coral

THE ELECTRONICS SYSTEM



Oblivion PCB serves as the central hub of the robot's system, integrating an ESP32 for data processing and motor control. Through an I²C multiplexer, it manages a gyroscope and four ToF sensors simultaneously, enabling precise and reliable navigation. The board includes interfaces for five servomotors, encoder inputs, and a transistor-based circuit for 12V LED PWM control. This design reduces wiring complexity and improves signal stability within a compact and efficient form factor.

THE PROCESS

It all began with a simple prototype built on a stripboard, including only the ToF sensors, gyroscope, I²C multiplexer, and ESP32. We selected 66 RPM DC motors to achieve an optimal balance between speed and torque, which is essential for handling slopes. The integrated encoders provide accurate real-time feedback, improving motion control and precision. The PCB also includes a "mustache" sensor used to detect material conductivity. All components were carefully chosen to reduce measurement errors while maximizing performance within budget constraints. For vision, we use the IMX219 camera module, positioned to capture the line at the center-bottom of the robot, ensuring greater stability and more reliable detection.

STATISTICS AND HISTORY



OUR FIRST VERSION

€ 500 €

7 MONTHS

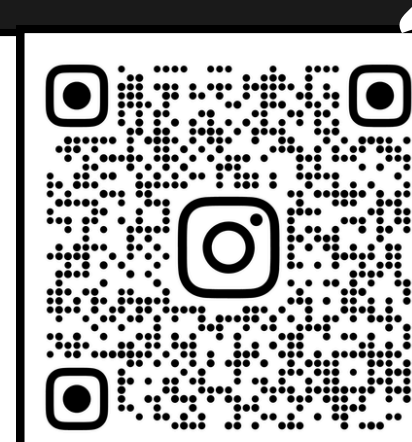
TEAM

Driven by a passion for advanced automation, **Oblivion Robotics** is a **student-led team from the heart of Sicily**. We bridge the gap between electronics, mechanics, and software **to push the boundaries of autonomous robotics**. By integrating high-performance hardware with sophisticated computer vision, **we have developed a resilient system designed to master the complex search-and-rescue challenges** of the RoboCup Junior 2026.



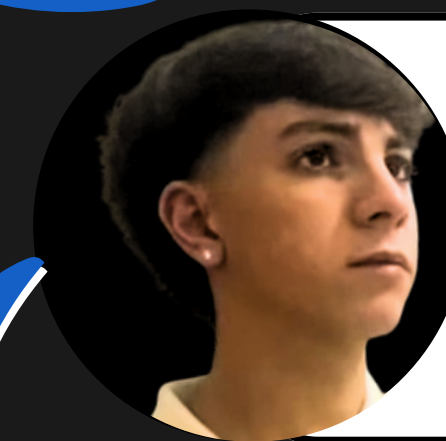
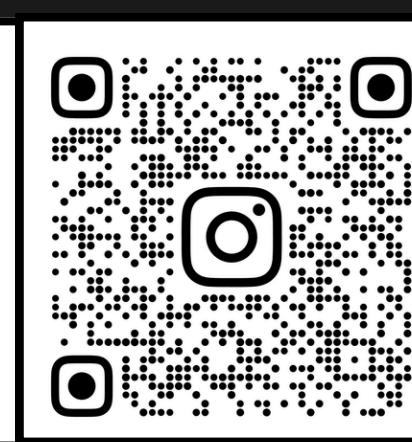
Mattia Gimmillaro

Leader and Software Engineer
17 y.o. - Sicily, Italy
I.T. Archimede Catania (CT)



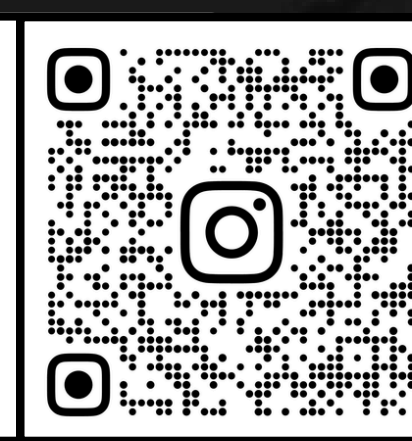
Emanuele Filetti

Electronics & Embedded Engineer
18 y.o. - Sicily, Italy
I.T. Archimede Catania (CT)

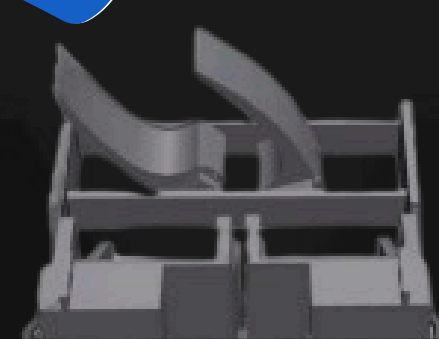


Gaetano Grasso

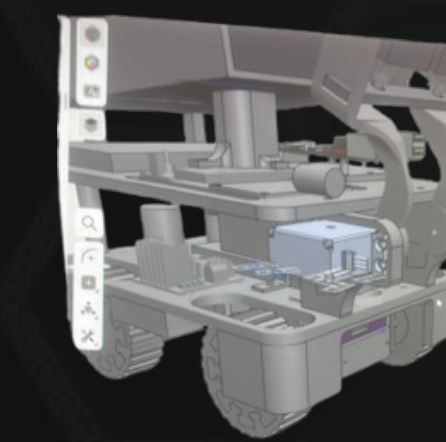
Mechanical Engineer
17 y.o. - Sicily, Italy
I.T. Archimede Catania (CT)



STRUCTURE & FLOWCHART

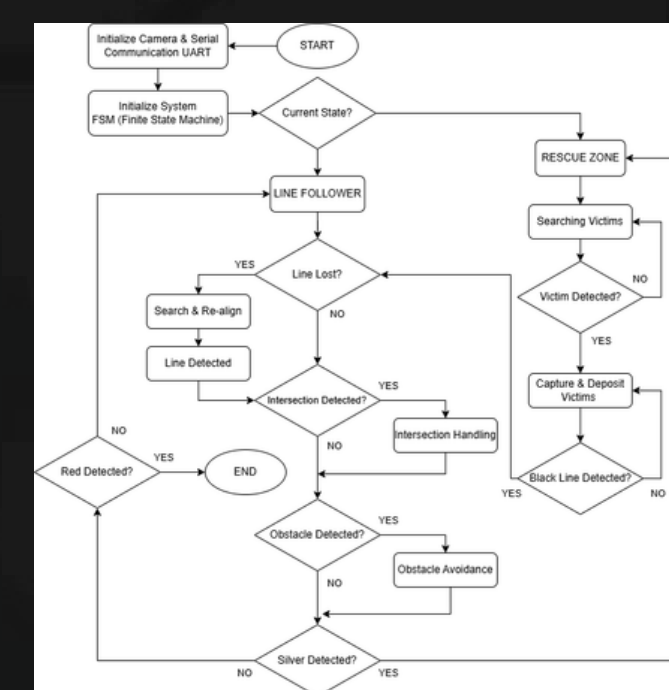


The chassis of the robot is designed and printed in 3D with Shapr3D.

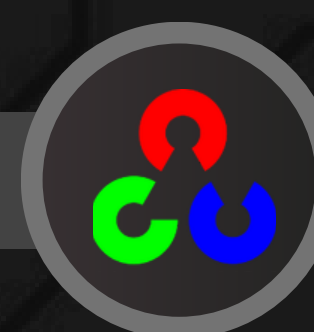


Another innovative solution is the ball collection system, which uses a decentralized clamping mechanism driven by two servomotors. This design allows the robot to guide and sort the balls efficiently into their respective containers.

FLOWCHART



SOFTWARE



THE SYSTEM ARCHITECTURE

4

The **Raspberry Pi** (Master) runs OpenCV vision and a **Finite State Machine** (Line, Obstacle, Rescue) to dictate robot behavior.

The **ESP32** (Slave) manages real-time hardware execution, using a dual-core FreeRTOS setup. **Both units synchronize over a high-speed UART Serial interface**, using a custom 11-byte binary protocol to ensure sub-millisecond latency.

```
match currentState:
    case "LINE":
        currentState = line_follower.run(cleanImage)
    case "RESCUE":
        currentState = rescue_zone.run(cleanImage)
    case "OBSTACLE":
        currentState = obstacle.run(cleanImage)
    case _:
        currentState = "LINE"
```

```
void setup() {
    commandQueue = xQueueCreate(10, sizeof(Command));
    // CORE 0: DEDICATED TO SERIAL I/O AND DATA TELEMETRY
    xTaskCreatePinnedToCore(serialTask, "SerialTask", 1024, NULL, 1, NULL, 0);
    // CORE 1: DEDICATED TO HIGH-PRIORITY REAL-TIME MOTOR CONTROL
    xTaskCreatePinnedToCore(motorTask, "MotorTask", 1024, NULL, 2, NULL, 1);
}
```

5

The robot **processes camera input using OpenCV**. For line following, the frame is divided into two regions and the black line is detected through contour extraction. A **delta value between the upper and lower centers of the line** is calculated, and its inclination is used to determine the robot's direction.

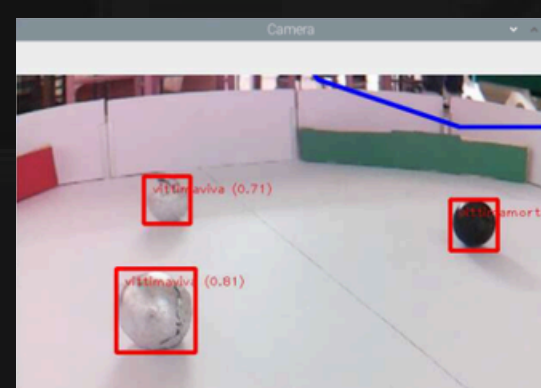


6

To detect intersections, a **green color mask is applied**. The system identifies the contour and computes its center; it then analyzes specific **Regions of Interest (ROIs)** above and on the sides of the marker to detect the black line's presence and determine the correct path.

```
rect = cv2.minAreaRect(greenContour)
box = cv2.boxPoints(rect)
box = np.int32(box)

if rectArea > config.MIN_GREEN_RECT_AREA:
    whitetop = cv2.countNonZero(cutTop)
    blacktop = cutTop.size - whitetop
    if whitetop > blacktop:
        curve = [1, 0]
```



7

In the rescue zone, victims are detected, classified, and tracked in real time using a **YOLOv8-based model** and their coordinates in the frame. The **Google Coral accelerator enables high-speed inference**, ensuring smooth and efficient processing.